

Humanoid Academy

Tag 2: Lococlient, SLAM



Unitree Martial Arts

<https://youtu.be/mUmlv814aJo>



www.ef-robotics.de

EF
ROBOTICS



Zeitplan Tag 2

Di 16.06.2026

08:30 Finden der Teilnehmer

09:00 Schulungsblock 1

10:30 Kaffee- und Teepause

11:00 Schulungsblock 2

12:30 Mittagessen

13:30 Schulungsblock 3

15:00 Kaffee- und Teepause mit Snacks

15:30 Schulungsblock 4

17:00 Ende Tag 2



Tagesziele

- Systemübersicht SSH, ROS ,Themen
- Mode Switching
- Loco Client und Navigation

Aufgaben:

- Abstrahierung Modus Wechsel
- Einfaches Bewegungsskript
- SLAM-Navigation



Systemübersicht G1

- ROS Umgebung
- Internetverbindung
- Sensoren (LiDAR / Kamera)



ROS2 Umgebung

- Ros2 foxy
- summary :
https://github.com/EF-Robotics-Humanoid-Academy/ef_ws/blob/main/g1/ros_envs/foxy/summary.md



RealSense Ansatz

- WLAN-Verbindung
- Streaming pipeline:
- Camera Server → Receiver



Grundlagen der Laufbewegungen

- Einführung loco client

https://support.unitree.com/home/en/G1_developer/rpc_routine

Value	Meaning	Comment
0	Static stand	Feet loaded; robot balancing but not stepping.
1	Dynamic / gait	Feet loaded; gait planner active (walking, etc.).
2	Feet un-loaded	Hanging or airborne.



G1 Programmierung

FSM Codes

ID	Name / Action	Typical use
0	Zero-Torque	Motors off – safe to handle legs.
1	Damp	Viscous damping only (soft) – default power-on.
2	Squat	Static low squat.
3	Sit	Sitting posture.
4	Stand-up	Firmware-defined stand, good after Damp.
501	Walking mode	Balance controller & gait planner.
802	Running Mode	Faster – less stable Locomotion

Development mode

< Service Status

● Functional ● Abnormal ● Close

ai_sport / 8.7.3.4 audio_player_service bashrunner / 1.0.3.1

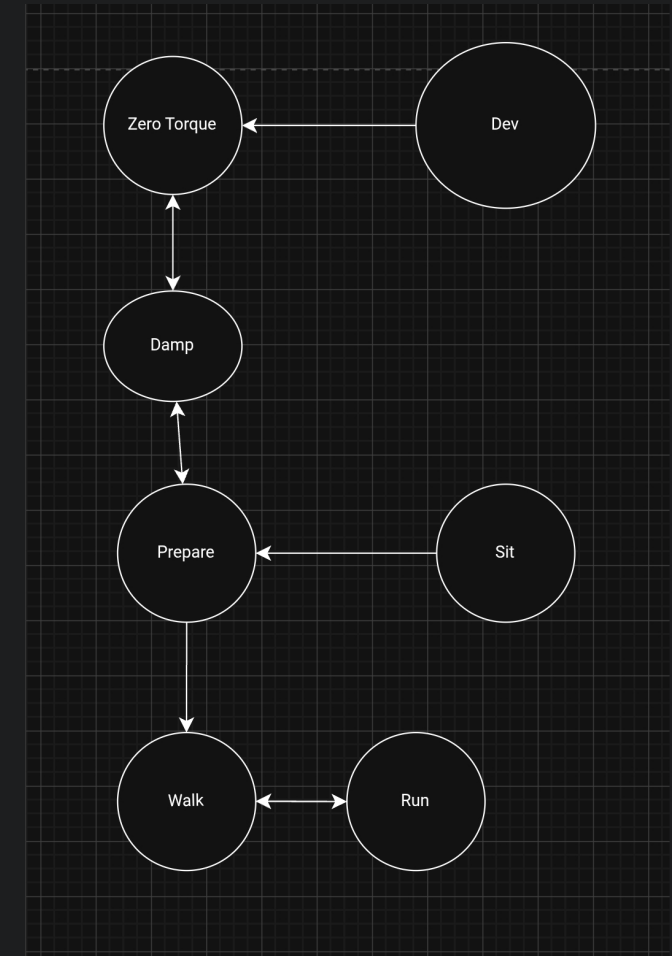
basic_service / 3.0.0.9 battery_guard / 1.0.0.9 chat_go / 1.9.2.73

dex3_service_l dex3_service_r emergency_stop / 1.0.0.4

g1_arm_example / 2.0.1.3 lidar_driver / 1.0.0.12 log_system / 1.0.0.13

motion_switcher / 1.0.0.1 net_switcher / 1.0.2.1 ota_box

robot_state / 3.0.0.7 robot_type_service / 1.0.2.3 state_estimator / 1.0.0.3



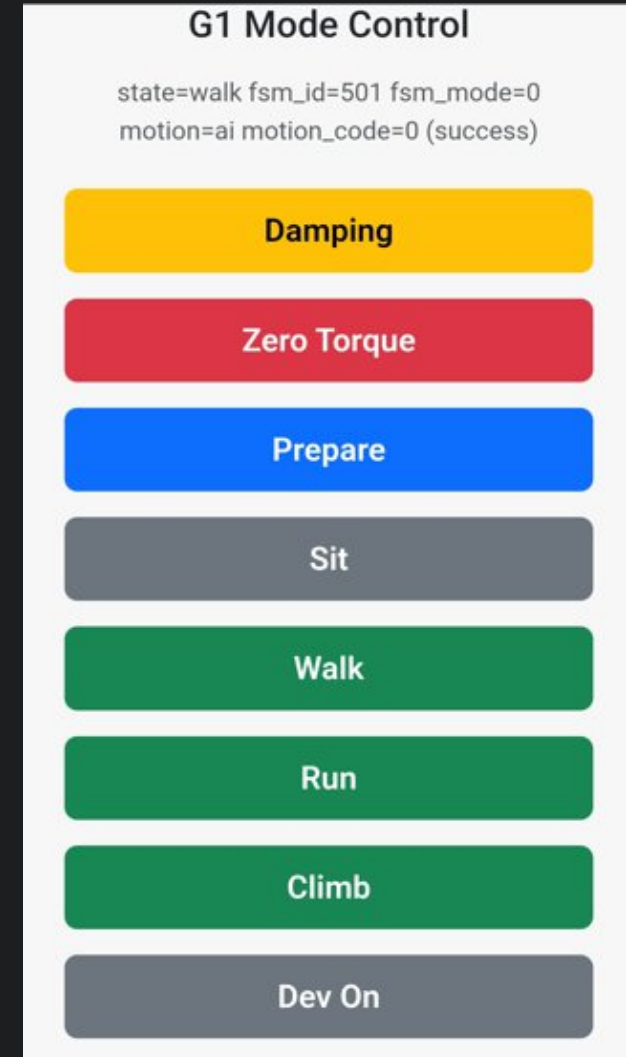
Aufgabe 1

- App oder Fernbedienung benutzen um den Roboter in einem Modus zu stellen
- Durch sdk oder Abstrahierungsmodule daten im Zustand zu sammeln (fsm, gaittype, velocities, joint poses, ...)
- Eigene Abstrahierungen für moduswechsel zu erstellen
- RGBD Zugriff über WLAN in eigenem PC



Aufgabe 1

- Im Jupyterlab ein Terminal öffnen
- Codex die sdk module durchlesen lassen
- Eine UI (Dash Webseite) für intuitiven Zustandswechsel erstellen



Grundlagen der Laufbewegungen

Einführung loco client

1) Minimale Imports

```
from unitree_sdk2py.core.channel import ChannelFactoryInitialize, ChannelSubscriber
from unitree_sdk2py.g1.loco.g1_loco_client import LocoClient
from unitree_sdk2py.idl.unitree_go.msg.dds_ import SportModeState_
```

2) Netzwerk- / DDS-Initialisierung

DDS einmalig initialisieren, bevor Clients/Subscribers erzeugt werden.

```
iface = "enp1s0"    # Netzwerk-Interface zum Roboter
domain_id = 0       # in der Regel 0
```

```
ChannelFactoryInitialize(domain_id, iface)
```

Grundlagen der Laufbewegungen

Einführung loco client

3) Loco-Client erzeugen

```
loco = LocoClient()  
loco.SetTimeout(10.0)  
loco.Init()
```

Danach sind Locomotion-APIs wie Move, StopMove, BalanceStand nutzbar.

5) Grundlegende Bewegungsbefehle

```
# Move(vx, vy, vyaw, continous_move=True)  
loco.Move(0.2, 0.0, 0.0, continous_move=True)  # vorwaerts  
time.sleep(1.0)  
loco.StopMove()
```

Hinweise: - vx: vor/zurueck (m/s) - vy: seitwaerts (m/s) - vyaw: Giergeschwindigkeit (rad/s)

Aufgabe 2

- LocoClient / Robot.move_for benutzen
- Roboter in einem U Verlauf laufen lassen:
 1. Roboter 1m nach vorne laufen lassen
 2. Roboter 90 Grad nach rechts drehen lassen
 3. Roboter 1m nach vorne laufen lassen
 4. Roboter 90 Grad nach rechts drehen lassen
 5. Roboter 1m nach vorne laufen lassen



Grundlagen der Laufbewegungen

Einführung loco client

7) Gait-Type-Steuerung

Wenn Firmware Gait-Umschaltung unterstützt:

```
loco.SetGaitType(0)    # Gehen / Balanced Gait  
loco.SetGaitType(1)    # Laufen / Continuous Gait
```

Empfohlenes Muster:

```
# Gehmodus  
loco.SetGaitType(0)  
loco.Move(0.2, 0.0, 0.0, continous_move=True)  
  
# Laufmodus  
loco.SetGaitType(1)  
loco.Move(0.4, 0.0, 0.0, continous_move=True)
```

Aufgabe Laufen

7) Gait-Type-Steuerung

Wenn Firmware Gait-Umschaltung unterstuetzt:

```
loco.SetGaitType(0)    # Gehen / Balanced Gait  
loco.SetGaitType(1)    # Laufen / Continuous Gait
```

Empfohlenes Muster:

```
# Gehmodus  
loco.SetGaitType(0)  
loco.Move(0.2, 0.0, 0.0, continous_move=True)  
  
# Laufmodus  
loco.SetGaitType(1)  
loco.Move(0.4, 0.0, 0.0, continous_move=True)
```

Wichtige ROS2 Topics

- /map → Generated map
- /odom → Robot movement
- /tf → Transform tree
- /scan → LiDAR data
- /camera/depth → Depth

```
/clicked_point  
/cmd_vel  
/goal_pose  
/initialpose  
/joint_states  
/parameter_events  
/robot_description  
/rosout  
/tf  
/tf_static
```

Sensor Skripte

- `rgb_d_cam.py`
- `imu_plot.py`
- `lidar_points.py`



Fazit

- G1 rviz simulation
- Slam concepts or arm motion



Repository Übersicht

- docs/
- REQUIREMENTS.txt
- scripts/sensors/



Workspace Struktur

- docs, scripts, modules
- G1 sport mode ROS
- simulation



Setup

❏ Bash

```
nano ~/.bashrc
```



❏ Bash

```
source ~/.bashrc
```

❏ Bash

```
# Source ROS 2 Foxy
```

```
source /opt/ros/foxy/setup.bash
```

```
# Source your workspace (change path if needed)
```

```
source ~/foxy_ws/install/setup.bash
```

</> Bash



```
colcon build
```

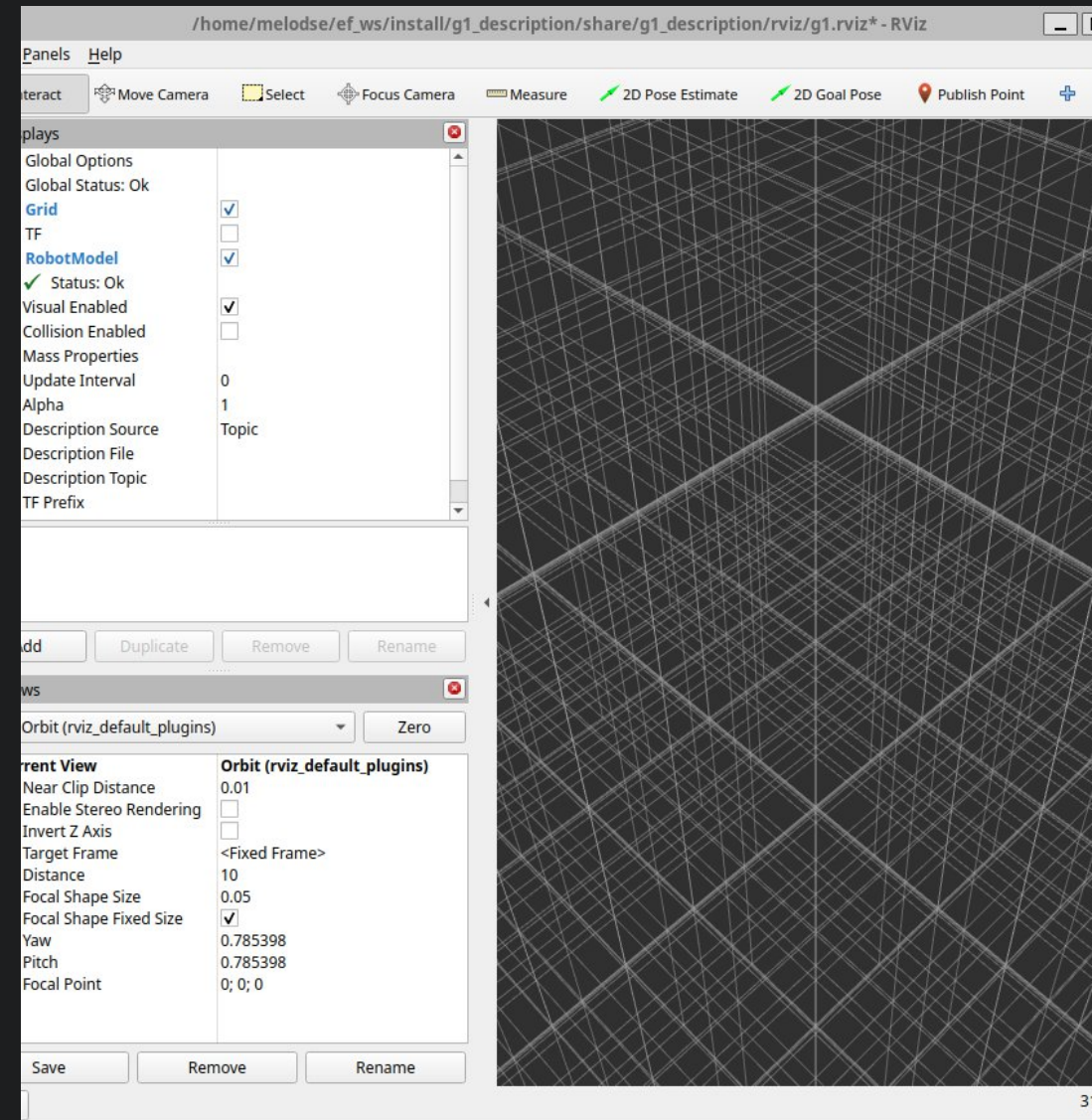
</> Bash



```
colcon build --packages-select package_name
```

RViz Setup (Schritt für Schritt)

- 1. Launch robot / simulation
- 2. Start SLAM node
- 3. Open RViz
- 4. Add displays



Topics:

Bash



```
ros2 topic list
```

If not correct:

Bash



```
ros2 daemon stop  
ros2 daemon start  
ros2 topic list
```

Still not working:

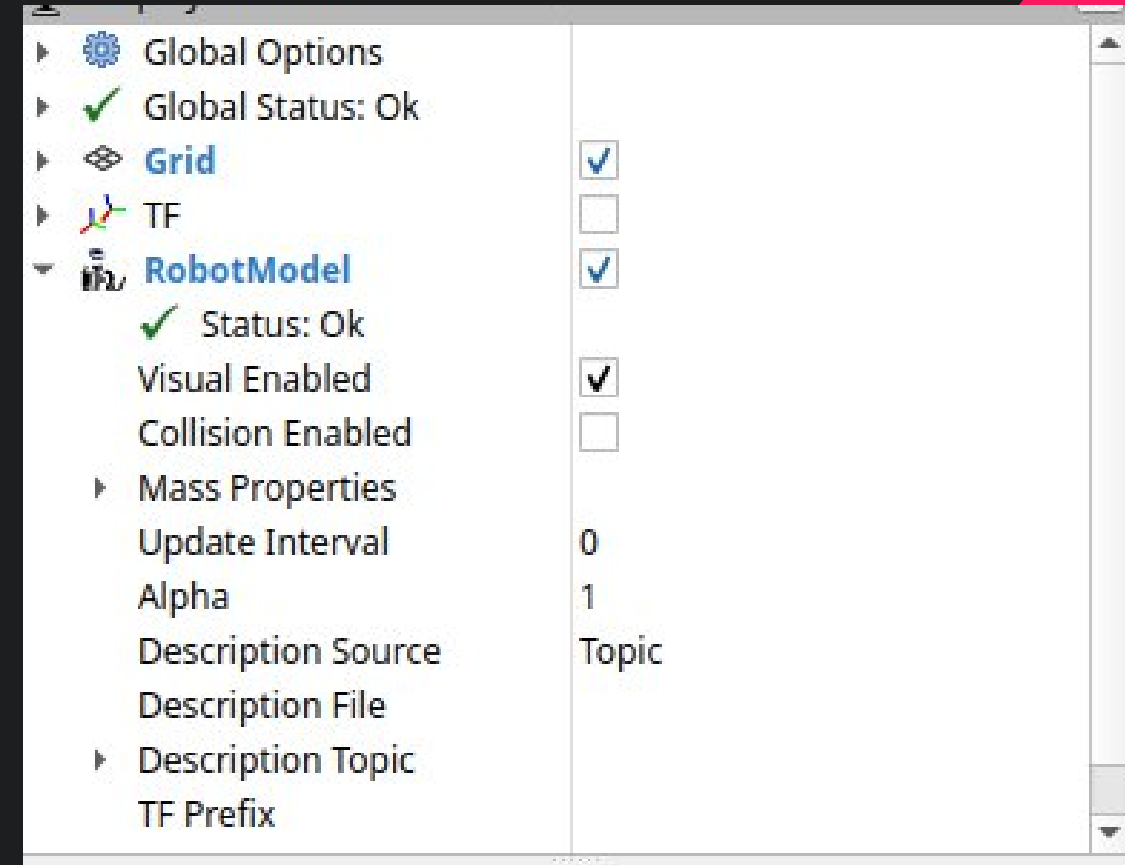
Bash



```
echo $ROS_DOMAIN_ID
```

RViz Anzeigen hinzufügen

- Map → /map
- TF → transforms
- LaserScan → /scan
- RobotModel → URDF



RobotModel

- ✓ Status: Ok
- Visual Enabled ☒
- Collision Enabled ☐
- Mass Properties
 - Update Interval 0
 - Alpha 1
 - Description Source Topic
 - Description File
- Description Topic **/robot_description**
- TF Prefix
- Links

Description Topic
Topic where filepath to urdf is published.

Add Duplicate Remove Rename

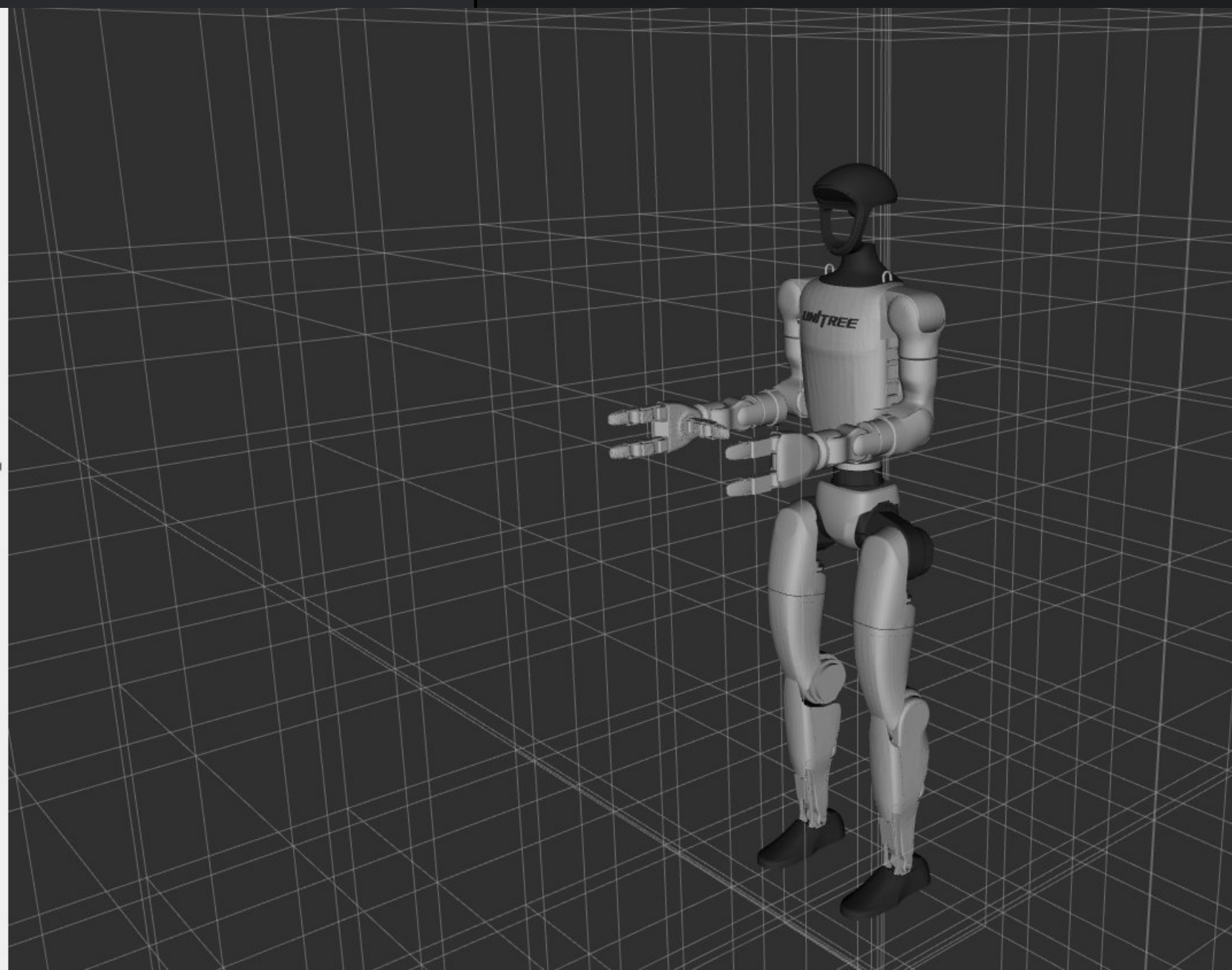
Views

Type: Orbit (rviz_default_plugins) Zero

Current View

- Near Clip Distance 0.01
- Enable Stereo Rendering ☐
- Invert Z Axis ☐
- Target Frame <Fixed Frame>
- Distance 2.51916
- Focal Shape Size 0.05
- Focal Shape Fixed Size ☒
- Yaw 0.815398
- Pitch 0.255398
- Focal Point 0; 0; 0

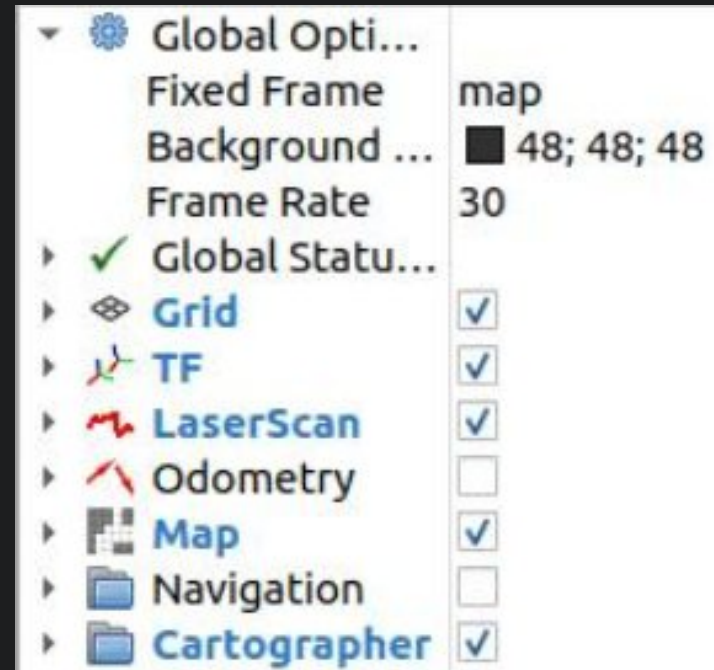
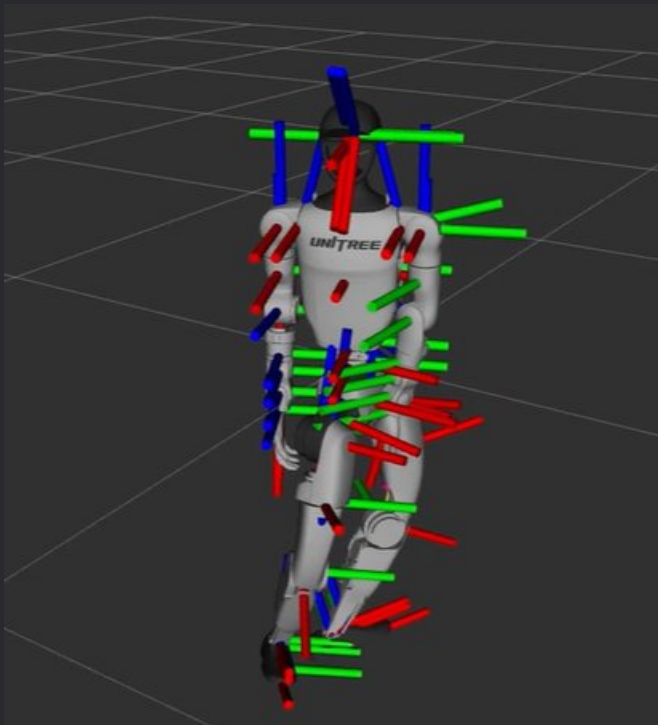
Orbit (rviz_default_plugins)



G1 RViz

Simulationsablauf

- Sensor Data → SLAM → Map
- Odometry → TF → Pose
- Everything visualized in RViz



Tanzen G1 simulation

Step 1: Set up a Conda virtual environment

```
conda create -n retarget python=3.10
```

```
conda activate retarget
```

Step 2: Install dependencies

```
conda install pinocchio -c conda-forge
```

```
pip install numpy rerun-sdk==0.22.0 trimesh joblib
```

Step 3: Run the script with parameters:

```
python rerun_visualize.py --file_name dance1_subject2 --robot_type [g1_23dof|g1_29dof|h1|h1_2]
```

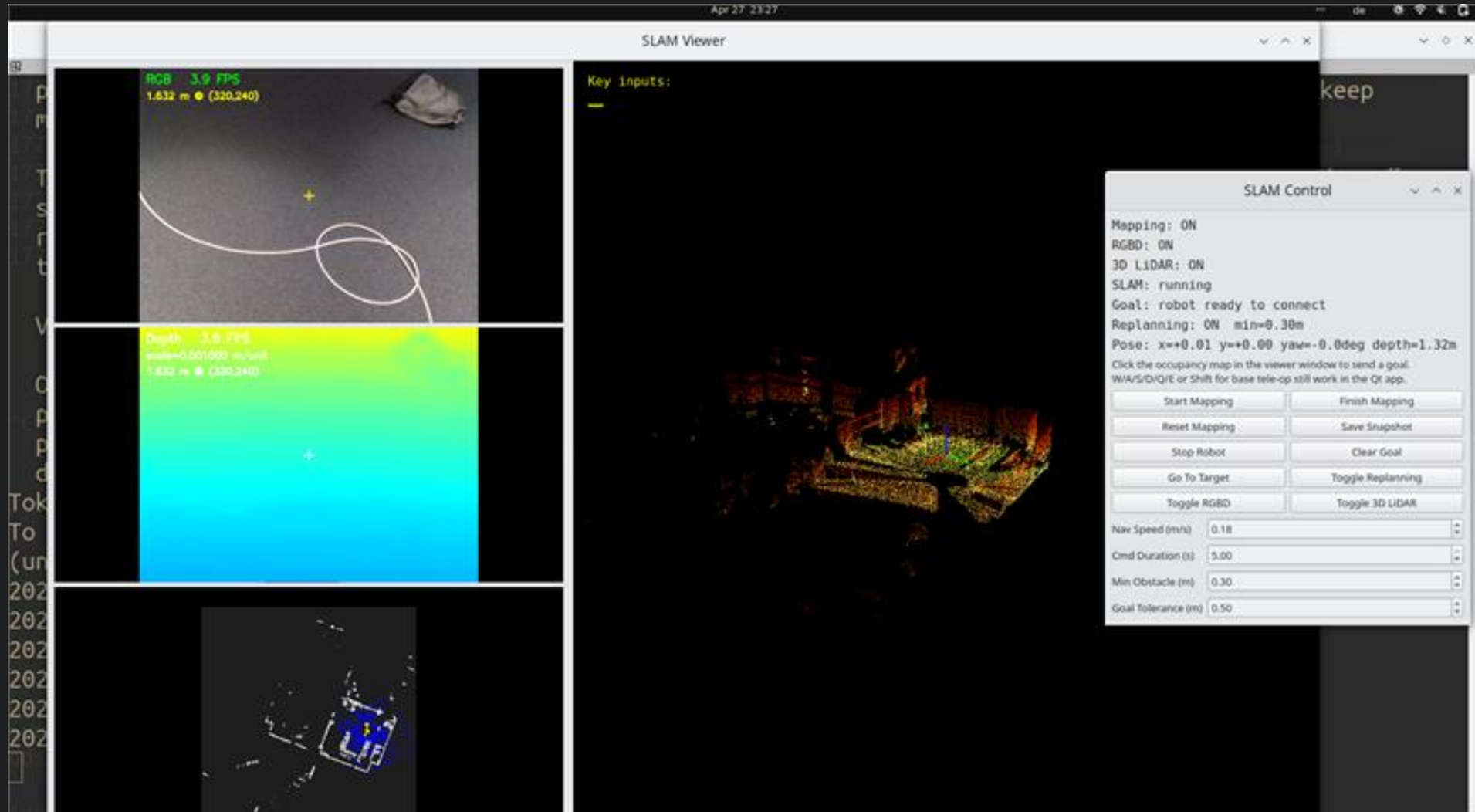


RViz Visualisierung

- Map → /map
- TF → transforms
- LaserScan → /scan
- RobotModel → URDF
- (Anzeige in RViz aktivieren)

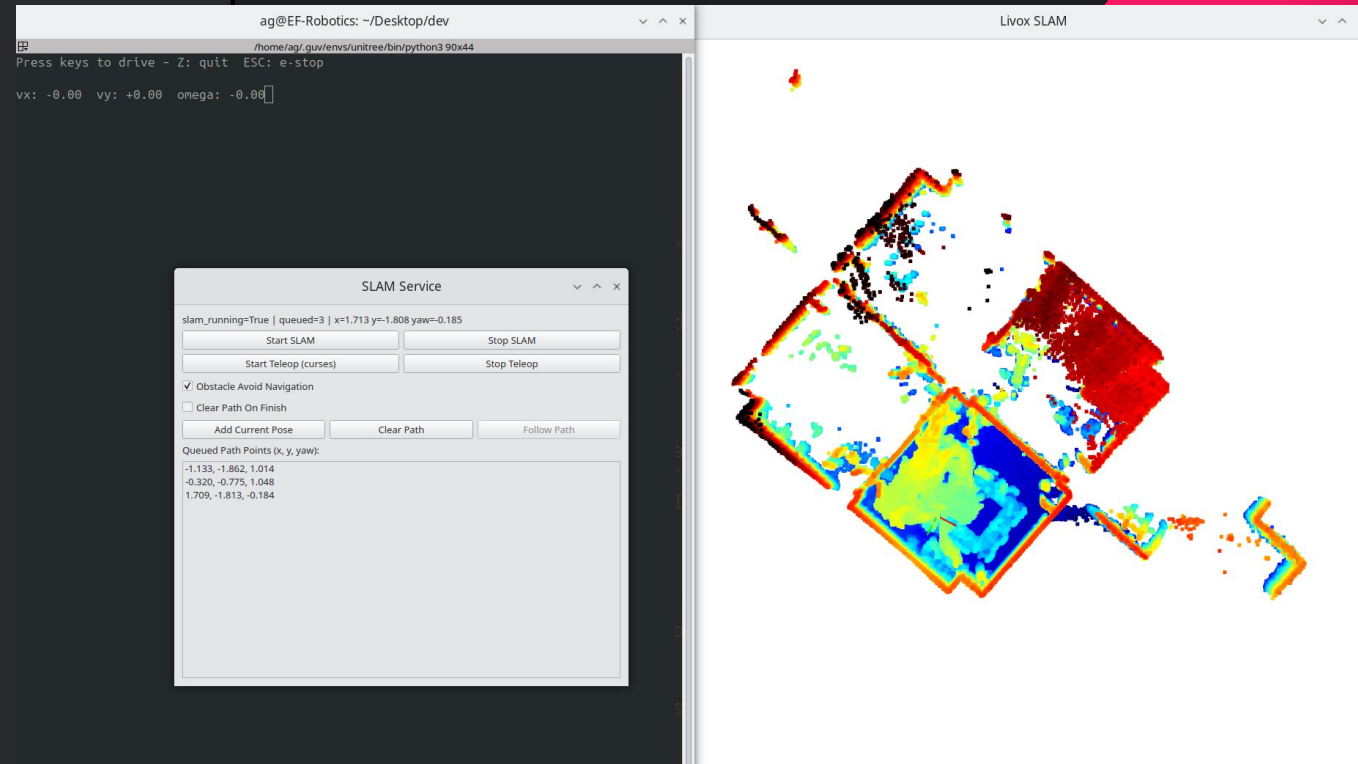


Kartierung und Navigation (SLAM)



Kartierung und Navigation

- Visualisierung
 - Erfordert den Livox Stack
 - https://github.com/EF-Robotics-Humanoid-Academy/ef_ws/blob/main/g1/dev/old_scripts/navigation/slam/slam_livox_doc.md



› in the slam notebook, add another cell that launches a modified version of the web app, where the user can provide a high level task such as `robot.face_wave()` or `time.sleep`, to be executed after reaching each point of the `execute_tasks` btn

Kartierung und Navigation Daten vom Roboter (Allgemein)

- Pfadplanung
- A* Algorithmus

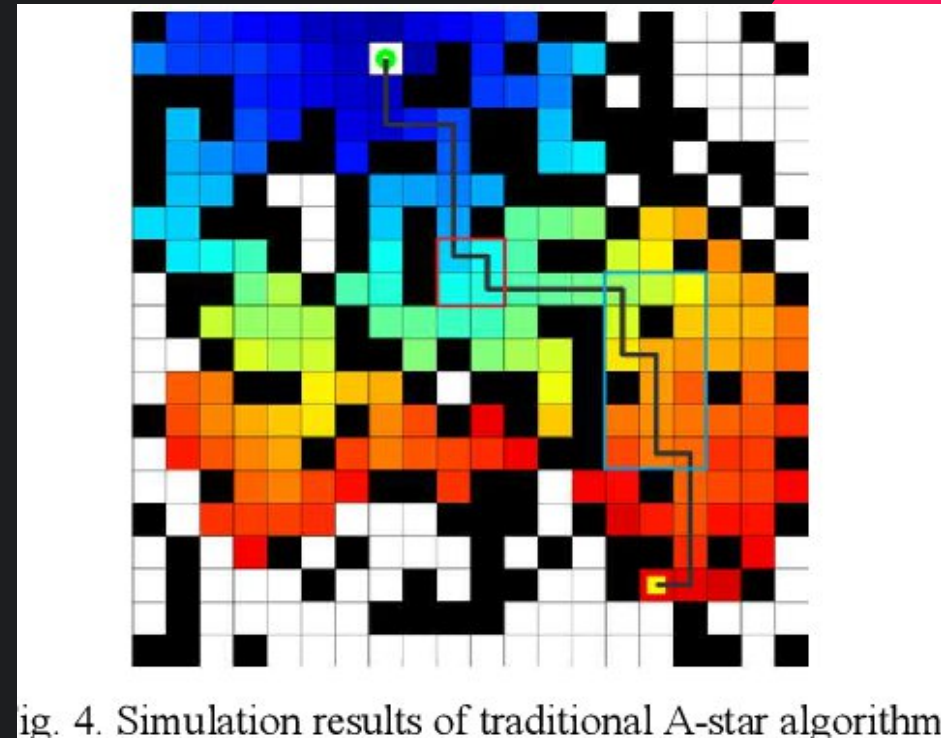
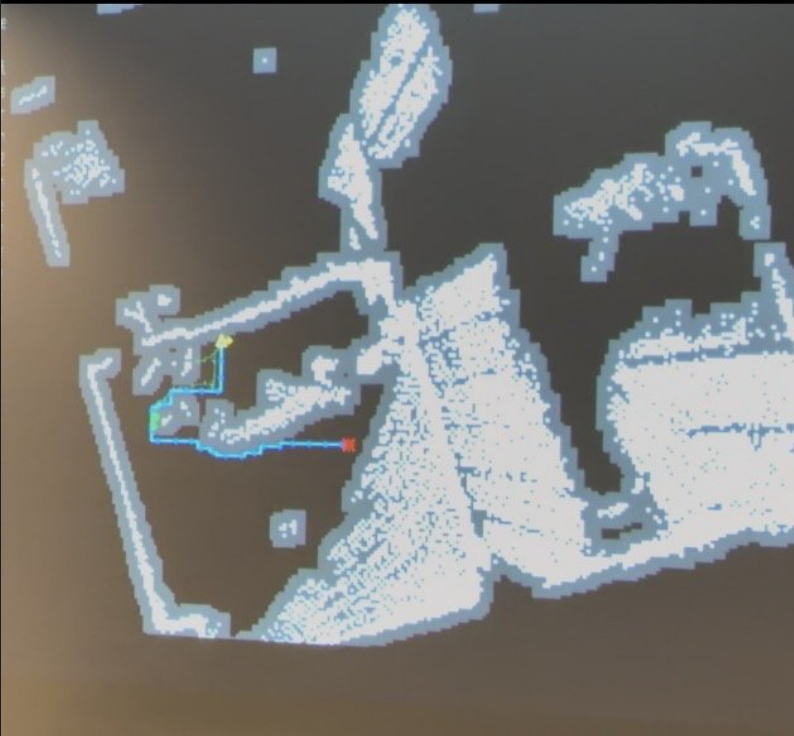


fig. 4. Simulation results of traditional A-star algorithm

Debugging Ablauf

- Kein Map → /scan prüfen
- Kein Roboter → /tf prüfen
- Keine Bewegung → /odom prüfen



Aufgabe 3

- Karte erstellen
 - SLAM starten
 - Roboter Bewegung
 - SLAM beenden
- Navigation
 - Lokalisierung
 - Pfadpunkte definieren
 - Navigation

Aufgabe 4

- Karte erstellen
 - SLAM starten
 - Roboter Bewegung
 - SLAM beenden
- Navigation
 - Lokalisierung
 - Pfadpunkte definieren mit vordefinierten High level Arm Bewegungen
 - Navigation

G1 Datenfluss

- /scan → SLAM → /map
- /odom → TF → Pose
- → RViz Visualisierung



Aufgabe 5

- Sdk or sdk modules um eine Navigationsfunktion zu erstellen mit:
 1. Benutzung von Livox sdk zur Visualisierung
 2. Zielpunkte definieren
 3. Navigation mit/Ohne Hindernisvermeidung



Grundlagen der Armtätigkeit

Gelenke

```
TOPIC_ARM = "rt/arm_sdk"
TOPIC_HAND_RIGHT = "rt/dex3/right/cmd"

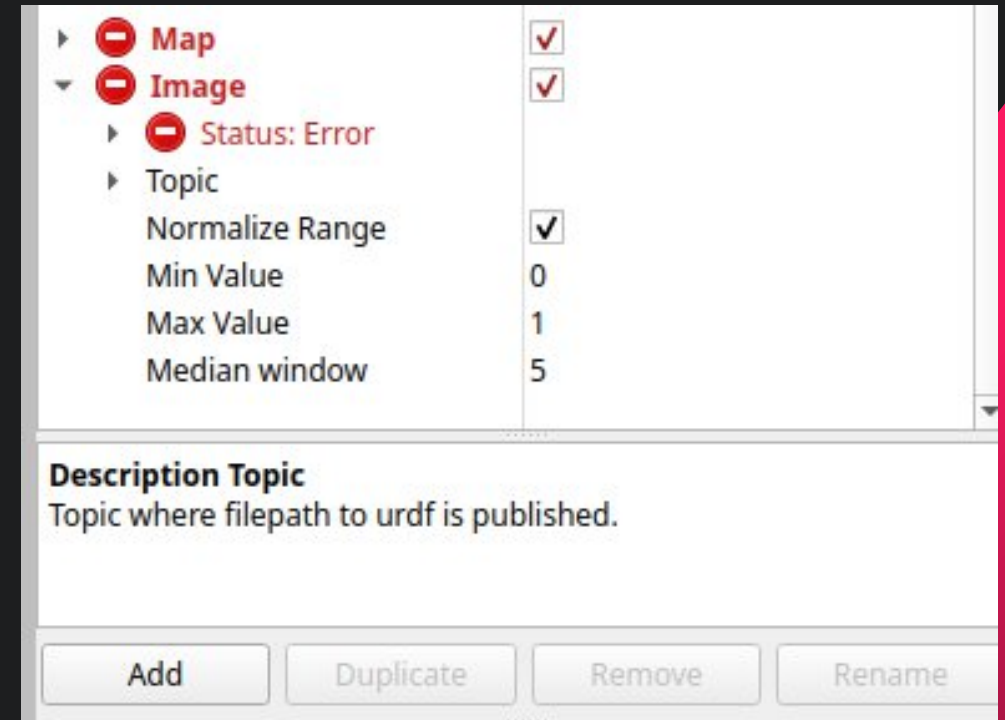
G1_NUM_MOTOR = 29
ARM_ENABLE_IDX = 29 # enable arm_sdk when q = 1 (per Unitree docs)

class G1JointIndex:
    WaistYaw = 12
    WaistRoll = 13
    WaistPitch = 14
    RightShoulderPitch = 22
    RightShoulderRoll = 23
    RightShoulderYaw = 24
    RightElbow = 25
    RightWristRoll = 26
    RightWristPitch = 27
    RightWristYaw = 28

ARM_JOINTS = [
    G1JointIndex.RightShoulderPitch,
    G1JointIndex.RightShoulderRoll,
    G1JointIndex.RightShoulderYaw,
    G1JointIndex.RightElbow,
    G1JointIndex.RightWristRoll,
    G1JointIndex.RightWristPitch,
    G1JointIndex.RightWristYaw,
]
```

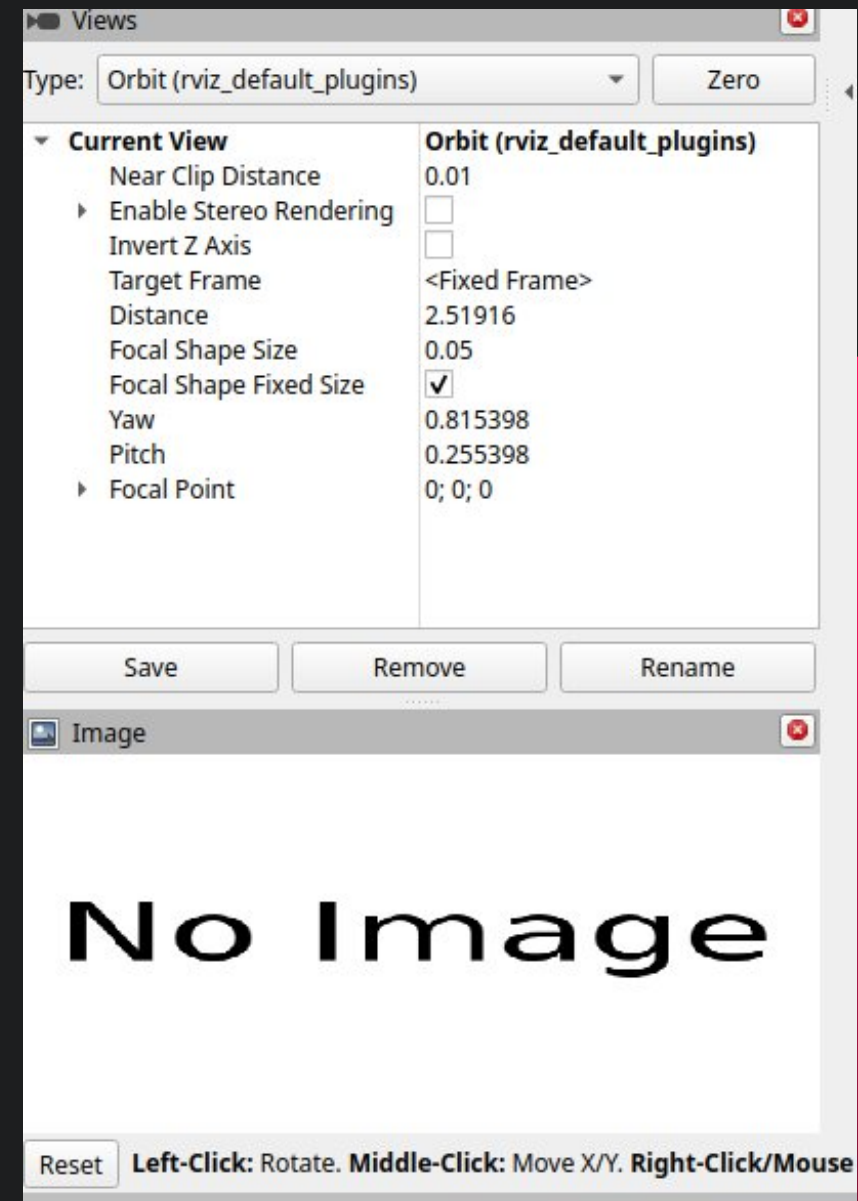
SLAM im G1 (Konzept)

- SLAM = Lokalisierung + Kartierung
- Der Roboter erstellt eine Karte und verfolgt seine Position
- Wird für autonome Navigation verwendet



ROS2 SLAM Pipeline

- Sensors (LiDAR / Camera)
- → SLAM Node
- → Map (/map)
- → Localization (/odom, /tf)
- → RViz Visualization



Überprüfungsliste

- Kartenaktualisierung in Rviz
- Roboter bewegt sich korrekt
- TF-Baum verbunden
- Sensorthemen aktiv



Häufige Debug-Probleme

- Keine Karte → Überprüfen /scan
- Kein Roboter → Überprüfen /
- Bewegung → Überprüfen /odom
- Verzögerung → Last reduzieren



Praktische SLAM Aufgabe

- SLAM ausführen
- Roboter bewegen (Skripte `test_motion`)
- Karte beobachten



Lernergebnis

- ROS Umgebung
- Moduswechsel
- High Level Locomotion
- SLAM und Navigation



Warum ist das wichtig?

- Autonome Navigation
- Hindernisvermeidung
- Industrie-Anwendungen (Logistik, Service-Roboter)



Live Demo Ablauf

- 1. RViz starten
- 2. Sensoren aktivieren
- 3. Karte entsteht
- 4. Roboter bewegt sich
- 5. Navigation A $\rightarrow$ B

